

Hierarchical GNN Message Passing for Node-Level Anomaly Detection in Industrial Control Systems

Shuaiyi Lyu^{ID}, Kai Wang^{ID}, *Member, IEEE*, Yang Liu, and Bailing Wang^{ID}

I. INTRODUCTION

Abstract—Advances in Graph Neural Networks (GNNs) have prompted remarkable progress in anomaly detection for securing the Industrial Control Systems (ICSs). As the core functioning block of a GNN network, message passing in most of the current frameworks is conducted via local aggregation, in which a node's vector representation is updated with messages from its directly connected neighbours. However, despite its efficiency over numerous application scenarios, such neighbouring aggregation mechanism tends to be highly biased towards a node's locality, and hence may not accurately profile the hierarchical semantics in layered ICS architectures, such as the supervisory relations among controllers and field devices. The resulting node embeddings, in this case, may not be knowledgeable enough to instruct downstream tasks such as fine-grained device-wise ICS anomaly detection. To address this issue, we introduce the Hierarchical Message Analyzer (the HMA), a new message passing scheme that explores a network's supervisory structural features and regulates a message's transmission paths to create balanced embeddings for node-level ICS anomaly detection. This model comprises in its architecture a Preprocessor that condenses the original data flow into initial node vectors, an Adjacency Parser that regulates how messages are transmitted in the aggregation process, an Encoder performing message passing in compliance with the adjacency info obtained from the Adjacency Parser, and a Decoder for label inference. We assess the HMA's performance over multiple evaluation metrics and compare it against various state-of-the-art baselines. Results on multiple datasets certify the HMA's validity and superiority in device-wise ICS anomaly detection.

Index Terms—Hierarchical message passing, meta-graph data collection, graph neural networks, anomaly detection, industrial control systems.

Received 27 November 2023; revised 12 October 2025; accepted 15 October 2025. Date of publication 20 October 2025; date of current version 12 March 2026. This work was supported in part by the Taishan Scholar Project of Shandong Province, China under Grant tsqn202408112, in part by the National Natural Science Foundation of China (NSFC) under Grant 62272129, and in part by the Key Laboratory of Cognitive Intelligence and Content Security, Ministry of Education, China under Grant RZZN202414. (*Corresponding authors: Kai Wang; Bailing Wang.*)

Shuaiyi Lyu is with the Faculty of Computing, Harbin Institute of Technology, Harbin 264209, China (e-mail: lvshuaiyi2568068@163.com).

Kai Wang is with the School of Computer Science and Technology, Harbin Institute of Technology, Weihai 264209, China, and also with the Shandong Key Laboratory of Industrial Network Security, Weihai 264209, China (e-mail: dr.wangkai@hit.edu.cn).

Yang Liu is with the School of Computer Science and Technology, Harbin Institute of Technology, Weihai 264209, China (e-mail: liuyang322@hit.edu.cn).

Bailing Wang is with the Harbin Institute of Technology (Weihai) Qingdao Research Institute, Qingdao 266109, China, and also with the Shandong Key Laboratory of Industrial Network Security, Weihai 264209, China (e-mail: wbl@hit.edu.cn).

Digital Object Identifier 10.1109/TDSC.2025.3623299

THE Industrial Control Systems (ICSs) are complex systems of heterogeneous devices dedicated for industrial process monitoring and control. They have been frequently targeted and attacked in recent years due to their increasing connectivity to public networks. In order to inflict massive physical damage or cause considerable economic loss, the majority of recent attacks aim at directly sabotaging the industrial processes supervised by the ICSs. As the operational status of industrial processes can be reflected in the numeric data streams of measurement values or operational states (See Fig. 1) with respect to each process-related ICS device (sensors, actuators, controllers, etc.), it is imminent to derive reliable schemes to detect anomalies in these streams in order to secure the industrial processes and avoid any potential hazards.

To discover malicious patterns in these streams, early methods such as the Deterministic Finite Automata [1], [2] series and the Statechart-based Detectors [3], [4] are proposed to intuitively detect anomalous snippets that deviate from the periodic characteristics of industrial processes. Lightweight and simple as they are, these methods usually suffer from an undesirably high false positive detection rate, in that the patterns extracted are usually simple and superficial while an actual measurement value tends to fluctuate frequently even in normal operating scenarios. This makes state machine generation difficult and therefore greatly weakens the methods' applicability to anomaly detection in dynamic ICS streams. In addition, lack of consideration of associativity among numeric streams with respect to different ICS devices further undermines the methods' detection accuracy.

To better exploit the complex variational semantics together with the correlational features of devices' numeric streams, neural network (NN) based solutions are introduced in numerous literature (e.g., [5] [6] [7]). By introducing layers of parameters, these methods conduct in-depth semantic learning over the input streams, produce in-depth features for anomaly detection and have, therefore, made decent improvements in detection accuracy. However, most of these methods perform graph-level state inference rather than conducting fine-grained device-wise adversary detection. This tremendously undermines the models' ability in finding which devices in the system an anomaly occurs in and severely impairs their practicality in ICS protection, as promptly locating and responding to anomalies is critical in securing the running industrial processes and preventing catastrophic outcomes such as physical damage and casualty [8], [9]. Furthermore, one cannot shut down the entire system as it may

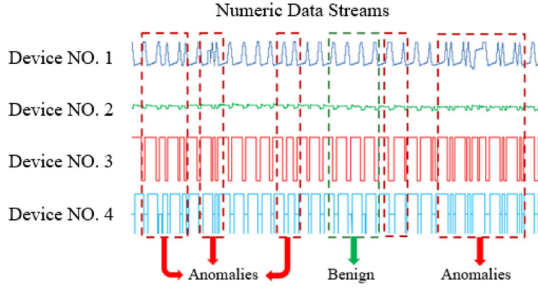


Fig. 1. Numeric data streams reflecting the operational status of industrial processes (Red-dashed blocks: Anomalous; Green-dashed blocks: Benign). Each stream corresponds to a measurement or operation index for a particular ICS device in a specified process (e.g. a sensor's measurement of liquid level inside some tank in a water treatment process, the operational mode of a pump (on/off) on a pipe, etc.).

cause considerable economic loss [10]. It will be very useful if operators can take actions over only the most relevant devices when an anomaly appears.

To timely discover the specific devices an anomaly impacts, Graph Neural Network (GNN) based approaches have been investigated. The GNNs are a powerful architecture for profiling graphs of arbitrary structures [11] and dealing with tasks on both coarse-grained graph level and fine-grained device level. The fundamental principle is to produce high-quality node embeddings via message aggregation for subsequent feature differentiation. However, how to design the message aggregation scheme so that the quality of information within the embeddings is good enough for device-wise ICS anomaly detection is a challenging technical issue that requires in-depth exploration. Much of the current literature is based on direct neighbourhood aggregation, in which a node obtains information from its immediate neighbours, causing embedding learning to bias towards a node's local surroundings. Simple as they are, when it comes to application in layered ICS architectures built upon heterogeneous devices, nevertheless, such aggregation schemes do not explicitly consider the supervisory relations among the devices. As a consequence, the embeddings produced by these methods are not an accurate or comprehensive reflection of how a node belongs to a hierarchical ICS topology. Therefore, these embeddings may not be knowledgeable enough to instruct downstream device-wise ICS anomaly detection tasks.

To address the aforementioned issue, we introduce the Hierarchical Message Analyzer (HMA). Distinguished from previous coarse-grained anomaly detection approaches producing graph-level results, our proposed HMA yields fine-grained labels on the node level that can be immediately utilized in tracking down the specific devices with which detected anomalies are associated. In addition, the HMA is a new message passing scheme that accurately profiles the hierarchical semantics in layered ICS structures via transmission path regulation. It generates balanced embeddings that reflect more precisely how specific nodes exist in the hierarchical context of an ICS architecture and, as a result, boosts the model's anomaly detection performance. The framework sequentially amalgamates the following components: (a) A Preprocessor that condenses the original numeric data streams captured from the ICS devices into downsized vector

representations featuring compressed temporal flow properties. These vectors are fed into the model as the initial feature vectors for the encoding process. (b) An Adjacency Parser that constructs the hierarchical architecture from the ICSs of interest and stipulates how messages flow during the aggregation process. (c) An Encoder that performs message passing based upon the new adjacency info produced by the Adjacency Parser and (d) A Decoder that maps the Encoder's output to the respective labels, which represent states of ICS devices for any applicable moment.

Our key contributions are summarized as follows:

- 1) Differentiated from the majority of current literature that conducts coarse-grained system-wise anomaly detection, we present a fine-grained anomaly detection framework producing device-wise labels that help locate the specific devices or components with which detected anomalies are associated with. As a result, our proposed framework is much more practical than current graph-level detection solutions in terms of anomaly retracing.
- 2) In order to accurately profile the core hierarchical semantics in typical layered ICS architectures that current neighbouring passing schemes fail to effectively capture, we develop the HMA, a hierarchical aggregation model regulating the message passing directions during the embedding generation process, to obtain a more precise perception of how specific nodes exist in the hierarchical context of an ICS architecture, than existing neighbouring aggregation paradigms in which embedding learning is biased towards a node's local surroundings. The resulting perception serves as a more balanced and reliable basis for fine-grained (node-level) state inference.
- 3) We implement the HMA over multiple datasets and evaluate its performance against numerous state-of-the-art baselines. Its effectiveness in ICS anomaly detection is demonstrated with various popular metrics.

The rest of the paper is structured as follows. Section II provides an overview of related work on ICS anomaly detection and basic GNN techniques. Section III defines the problem and illustrates the general principles of the proposed framework. Section IV provides a detailed description of the model architecture. Section V summarizes the experiment procedures, results and analysis and Section VI concludes the paper.

II. RELATED WORK

In this section, we summarize the literature in ICS anomaly detection and the Graph Neural Networks.

1) *Anomaly Detection*: As security issues in the ICSs have drawn substantial attention over the past decade, there has been numerous reviews discussing typical structures of ICS networks [12], as well as the security challenges they are confronted with [13], [14]. In light of the commonality of these reviews that there exists prominent consistency and periodic features in ICSs' communication patterns and process-oriented semantics, early traditional methods [15], [16], [17] are introduced to detect anomalies via periodicity mining, using methods ranging from classic statistical analytics [17] to state automata generation [1], [2]. Such solutions are favourable in situations where the

inspected communication is relatively simple so that the quantity of states remains manageable. Nevertheless, they suffer from a rather large false detection rate when handling flows exhibiting complex semantics, not to mention their inapplicability over relational characteristics.

To tackle this challenge, machine learning approaches are investigated and developed [18], [19]. Early contributions such as the k -NN methods [20], although simple, are still proved efficient in classification tasks and scalable to large datasets. Then, in order to discover high quality non-linear characteristics more accurately, the Robust Deep Autoencoder (RDA) [21] and the DAGAN [22] are introduced. More recently, intrusion detection systems are created [6], [7], [23], [24] via Convolutional Neural Networks, Long-Short Term Memory Networks or their combinations.

Apart from the aforementioned schemes, applications of unsupervised methods in anomaly detection are also prevailing. Relevant solutions include outlier detectors such as One-Class SVM [25] [26] and Isolation Forest [27], [28], [29], clustering methods including k -Means [30] and their variations, as well as neural network based models [31], [32], [33], [34]. For example, an efficient NN-based structure coupled with a PCA block [34] is developed for detecting anomalies by leveraging the data streams' time and frequency characteristics.

Last but not least, there are numerous other NN-based methods developed via transfer learning [35] and federated learning [36].

While these methods claim superior results, most of these approaches, to the best of our knowledge, are specifically designed for system-wise (graph-level) state inference rather than a more fine-grained device-wise (node-level) scenario.

2) *Graph Neural Networks*: To derive schemes capable of manipulating irregular data, Graph Neural Networks have been developed and applied to a great many domains of research and applications, including anomaly detection in Industrial Internet of Things (IIoT) [11]. Relevant work features multiple categories including but not limited to graph topology learning, message passing or aggregation, decoding mechanism and so forth, with respect to different functioning components of a GNN model.

The main goal of graph topology learning is to create or improve the quality of graphs used for subsequent representation learning. To achieve this, unsupervised learning methods via adaptive graph learning (BAGE) and variational adaptive graph learning (VBAGE) [37] are proposed, with which adjacency matrices are updated dynamically via training instead of being configured as a static input. The Graph Deviation Network (GDN) [38] and the Transformer-based framework (GTA) [39] adopt similar ideas in determining the correlational topology used in model training to address adversary detection issues on the graph level. In addition, the Fused Sparse Autoencoder and Graph Net (FuSAGNet) [40] are designed to explicitly model the relational patterns within the multivariate time series in order to achieve system-wise anomaly detection. More recently, the GSLCN [41] and TPGraph [42] are developed to respectively investigate the effect of long(short)-term dependency and feature correlation mining on a graph's contextual expression ability.

In terms of message passing or aggregation, available solutions in the current literature mostly fall in the neighbouring aggregation paradigm, in which a node obtains knowledge from its direct neighbouring nodes. Extensive methods have been proposed in this category including the GCN [43], TAGCN [20], GAT [44], GraphSAGE [45], etc. Apart from these methods that aggregate information from units of nodes, alternative message updating techniques based on meta-graph aggregation [46] have been demonstrated effective in dealing with semi-supervised learning problems on attributed heterogeneous information networks. Furthermore, in order to improve the model's ability in graph-level classification, hierarchical graph learning [47] and pooling approaches [48], [49] are developed. There are also trials in considering edge profiles in deriving comprehensive embeddings [50] for subsequent tasks.

There is abundant work in the literature that highlights other respects of a GNN process as well. For example, a decoding technique employing global-local integration (GLIN) [51] is designed to incorporate both a node's local identity and its global awareness in ICS anomaly detection. The FT-GCN [52] is suggested as a multi-view trial in traffic adversary detection. The feasibility of decoupling node representation learning and state classification [53] in training is investigated. Preliminary studies show that decoupled training indeed tends to outperform joint training, but its performance may deteriorate provided that the behavioural patterns and label semantics are highly inconsistent.

A brief summary of current literature is displayed in Table I.

As suggested in Section I, device-wise anomaly detection is important for ICS process protection, as operators cannot terminate the whole industrial process at the risk of huge financial loss. It would be extremely helpful if anomalies can be located so that operators may take appropriate actions over specific nodes without interrupting the rest of the system. Thus, GNN-based schemes are adopted for anomaly detector design in this paper, as they naturally deal with fine-grained tasks and yield node-level outputs. Particularly, to create node embeddings that better reflect the supervisory semantics in specified ICS scenarios, we introduce a meta-graph-based hierarchical message passing fashion (HMA) that breaches the convention of neighbourhood message aggregation. Our proposed method is designed to operate on ICS architectures in which the associativity among homogeneous devices (such as all sensors) can be completely unknown. Its conceptual distinctiveness as well as advantages over existing meta-graph based or hierarchical learning approaches can be stated as follows:

Differentiated from current hierarchical learning methods, the HMA can be viewed as a path regulator defining how node information flows in message aggregation without altering the original graph, while existing pooling approaches such as Diff-Pool [49] and HGP-SL [48] are graph feature condensers that integrate node features via graph streamlining. These pooling approaches are not applicable in device-wise anomaly detection, as they are designed to produce graph-level embeddings and do not generate representations for individual nodes in the graph. Compared to current meta-graph-based mechanisms, the HMA is a hierarchical feature extractor that operates on layers

TABLE I
GNN-BASED WORK SUMMARY

Category	Focus	References	Features
Reviews	Technical analysis	[12] [13] [14]	ICS structure and security challenges, etc.
Traditional methods	Periodicity mining	[1] [2] [15] [16] [17]	Statistical analytics, state automata generation, etc.
NN	In-depth feature learning	[23] [6] [18] [19] [20] [21] [22] [7] [24]	High-quality non-linear characteristics mining, etc.
NN	Unsupervised learning	[25] [26] [27] [28] [29] [30] [31] [32] [33] [34]	Outlier detection, clustering, etc.
NN	Other contributions	[35] [36]	Transfer learning, federated learning, etc.
GNN	Graph learning	[37] [38] [39] [40] [41] [42] [47]	Adaptive learning, relational pattern modeling, dependency, feature correlation, etc.
GNN	Message aggregation	[43] [20] [44] [45] [46] [50] [48] [49]	neighbouring & meta-graph aggregation, hierarchical graph learning, edge aggregation, etc.
GNN	Other contributions	[51] [52] [53]	Decoding, multi-view, decoupled learning, etc.

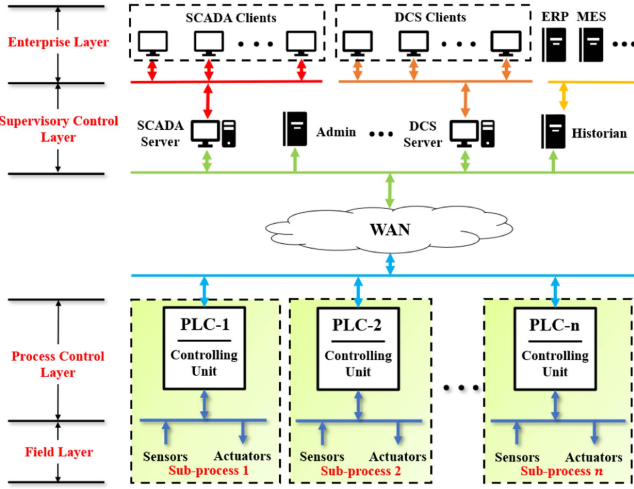


Fig. 2. ICS Architecture.

constructed from specified meta-graphs, while existing methods such as meta-graph aggregation [46] are local information integrators that aggregate neighbouring features from randomly produced sub-graphs. Current meta-graph aggregation method may not be an optimal option for ICS anomaly detection, as they fail to highlight the supervisory structural properties a typical ICS network renders, for which the produced embeddings are not an accurate reflection of how a node exists in the context of a layered ICS architecture.

III. PROBLEM STATEMENT AND MODEL DESIGN

In this section, we formulate the problem to be addressed and comprehensively introduce the concept and logic of the proposed model.

A. Problem Statement

Typical ICS networks are generally stacks of explicitly-differentiated functional layers, each of which has its own designated role to play. The architecture of a regular ICS network is illustrated in Fig. 2. It comprises a field layer with numerous (typically hundreds or thousands of) sensors and actuators that directly interact with the core industrial processes, a process control layer incorporating the controlling devices supervising the sensors and actuators in the field layer, a supervisory control layer and an upper enterprise layer for process management,

control, data storage, etc. Its top layer are usually accessible via public network for remote management.

One of the aspects differentiating the ICSs from regular networks lies in the vulnerability of the underlying physical processes. Anomalies in ICSs are not only triggered by the wide range of cyber attacks on the network level, but may also result from the sneaky attempts in breaching the regular system operating convention, which potentially leads to chaotic states in the physical processes. Specifically, based on [54], our assumed threat model (resources, goals and capabilities) for ICSs can be described as follows:

1) *Resources*: An attacker may have access to specialized tools to penetrate an ICS's enterprise layer or supervisory layer systems from external environment such as the Internet. He may also be an employee or ex-employee (e.g. a system operator) or be in connection with internal staff who have opportunities to damage the target ICS systems. In addition, it is likely the attacker is knowledgeable in relevant industrial fields and is aware of an ICS's potential vulnerability that can be exploited for hazardous purposes.

2) *Goals*: The main goal of an attack to an ICS is to impair its availability, which may lead to a) interruption or termination of physical processes that causes huge economic loss and/or nationwide instability, b) damage to critical infrastructure that are hard or impossible to recover so as to impede industrial development, c) severe casualty that gives rise to chaos in societies.

3) *Capabilities*: Attacks against an ICS physical process may exhibit the following capabilities: a) sensor data theft and/or distortion, b) actuator command listening and/or modification, c) direct physical penetration and/or control corruption that causes abnormal changes in sensor readings or actuator mode sequences (e.g. sensor measurement of liquid acidity or pump operational mode in water treatment process). Means of execution include but are not limited to remote network infiltration, USB importation, on-site human operation, etc.

Apart from data theft and command listening, all other capabilities of an attack targeting ICS processes presumably lead to abnormal deviations in device readings. Therefore, discovering such deviations is crucial in protecting the ICS processes and circumventing potential infrastructure damage and economic loss.

In this work, the term anomaly corresponds to an abnormal deviation in some device reading feature for a specified physical process, and the general Anomaly Detection problem is defined as below:

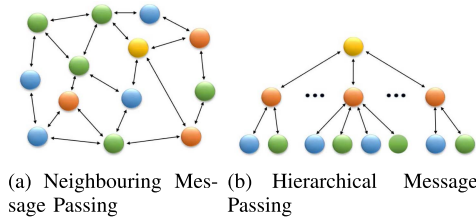


Fig. 3. Message Passing Scenarios.

Definition 1: Given an industrial field network $G(V, E, X(t), Y(t))$, where V , E , $X(t)$ and $Y(t)$ correspond to sets of the network's devices, connections, device reading features and ground-truth states at time t , respectively, find model $M(V, E, X(t)) = \hat{Y}(t)$ such that for $\forall t, \hat{Y}(t) = Y(t)$. Note that the term "state" can be an n -ary label representing the condition of a particular device, such as 0 for Normal and 1 for Anomaly given $n = 2$, or 0 for Normal and $k \geq 1$ for anomalies of different categories when $n \geq 2$. Simply put, our goal is to create a model that maps all normal and anomalous device reading patterns to the correct categories.

B. Model Design Principles

This work proposes a hierarchical GNN approach to address the aforementioned problem by applying a new message passing mechanism to node embedding learning. Unlike the classic GNN implementations that prescribe complete adjacency interaction over the graph regardless of the nodes' diverse contextual semantics, we concentrate on establishing an order that stipulates message redirection based on the specific role assigned to each node. We exemplify this concept with the following data collection scenarios.

Data collection occurs frequently in populated communities. One member (or collector) initiates the process, receives and analyzes the information obtained over the whole community before generating a feedback broadcast. The approach employed for data collection tends to dramatically influence the process's legitimacy and efficiency. Two scenarios are presented as follows for demonstration (See Fig. 3).

a) Neighbouring Message Passing: Members are arbitrarily mingled and exchange knowledge with their closest neighbours up till the point when info from each member has been successfully delivered to the collector. In this case, all individuals operate in the same manner despite their role difference.

b) Hierarchical Message Passing: Rather than being equally treated, members are arranged into groups and sub-groups in which a supervisor is designated for collecting and transferring group info as well as receiving feedback from their supervisors in larger groups. This creates a well-organized hierarchical structure enabling multiple collecting processes to occur simultaneously, allowing for a potential boost in terms of data completeness as well as operation efficiency for large communities.

The ICSs are heterogeneous networks with a multi-layered topology, which can be abstracted into graphs that current convolutional models such as the GCN and GAT can easily handle. Nevertheless, given the supervisory characteristics of

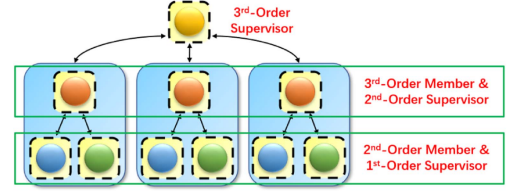


Fig. 4. Meta-Graph Definition.

the industrial control processes, these state-of-the-art models may not effectively capture the superior-subordinate semantics over the context of the graph, due to the fact that the neighbouring message passing fashion, as shown in scenario a) above, does not distinguish the roles each node plays. Therefore, we strive to discover alternative embedding learning methods that promote organized message aggregation in hierarchical graphs, as illustrated in scenario b).

We investigate a recursive hierarchical message passing scheme that leverages the properties of a layered structure and construct message aggregation blocks within the GNN layers via meta-graph data collection.

Definition 2: A meta-graph G_m of graph $G(V, E, X(t), Y(t))$ comprises a set $V_m \subseteq V$ of semantically linked nodes and a set $E_m \subseteq E$ of edges on condition that (i) for $\forall v \in V_m, \exists \tilde{v} \in V_m \setminus \{v\}, (v, \tilde{v}) \in E_m$ and (ii) for $\forall (v, \tilde{v}) \in E_m, v \in V_m$ and $\tilde{v} \in V_m$.

In a typical industrial network, field devices are distributed into distinct controlling cycles over separate regions. We define a series of meta-graphs based on these functional and geographical relations, as illustrated in Fig. 4. These meta-graphs are arranged by their structural complexity in an ascending order. They're interpreted as components of the network on different scales. The single-node meta-graphs in the black dashed blocks are the simplest patterns in the graph architecture. Hence they serve as a basis for construction of succeeding meta-graphs in the series. They represent single devices in the network such as a sensor or an actuator, etc. The triple-node meta-graphs in the blue-shaded patches correspond to double-layered relationships such as an individual controlling cycle. They consist of a controller, a sensor and an actuator measuring and regulating a specific physical state. These cycles contribute to the more complex topology (the entire Fig. 4), which can represent a department at a certain location operating over multiple controllers.

As illustrated in Fig. 4, each complex meta-graph consists of a supervisory node (the top node) and a number of meta-graphs whose order is precisely one less. The term order i denotes the position of each meta-graph $G_m^{(i)}$ in the list of all meta-graphs $[G_m]$ sorted by structural complexity. Our proposed message passing scheme is defined as an ordered sequence of 3 events: message aggregation, message exchange and message feedback. In message aggregation, all meta-graphs except the largest one, propagates their messages to the supervisory node of the meta-graph with the next higher order. For example, in an ICS network where controllers are appointed the 2nd ordered supervisory nodes (the orange nodes), sensor and actuator nodes (the blue and green nodes) swap messages directly with the controllers.

Then the controllers aggregate all the received messages, create condensed meta-graph representations and report them to the supervisory node (the yellow node at the top) of the 3rd ordered meta-graph, which might be a gateway, a supervisory machine, or simply a dummy node without any role, etc. Upon reaching the point where there are no higher order meta-graphs applicable, the message aggregation process is complete and the highest order supervisory nodes at the current state conduct message exchange and echo the resulting condensed messages back to the lower level meta-graphs along exactly the same paths used for message aggregation. The latter process, denoted as message feedback, is identical to the message aggregation procedure, except it works in the opposite direction.

In order to implement the aforementioned procedures, a layered structure is constructed in which the supervisory nodes of all the highest order meta-graphs are aligned in the top layer, and that all other nodes are assigned to their respective positions in the structure based on their distances to these highest order supervisory nodes. Details of the layer alignment process are explained in Section IV-B. When the structure is ready, messages produced at the bottom structural layer are elevated to the top, updated at the top and reverberated to the bottom. By the time all nodes on the bottom structural layer have received an echo from the top, one round of message passing process comes to an end and the resulting vectors are fed into the GNN's $UPDATE(\cdot)$ operator. To further explain it using matrix representations, suppose A is the original graph adjacency matrix and $H^{(l)}$ the updated node embeddings after the l -th round of message passing. In conventional GNN frameworks, message passing is conducted as follows,

$$H^{(l+1)} \leftarrow \tilde{A}H^{(l)}W. \quad (1)$$

where $\tilde{A}$ is the normalized adjacency matrix of A . In our approach, however, this is achieved via (2) and (3), with (2) corresponding to message aggregation, and (3) the message exchange and feedback steps.

$$H^{(temp)} \leftarrow \prod_{i=1}^{n-1} \tilde{A}^{(n-i)(n+1-i)} H^{(l)}. \quad (2)$$

$$H^{(l+1)} \leftarrow UPDATE \left(\left(\prod_{i=1}^{n-1} \tilde{A}^{(i)(i+1)} \right) \tilde{A}^{(n)} H^{(temp)} \right). \quad (3)$$

where

$$\prod_{i=1}^{n-1} \tilde{A}^{(n-i)(n+1-i)} \leftarrow \tilde{A}^{(n-1)(n)} \tilde{A}^{(n-2)(n-1)} \dots \tilde{A}^{(2)(3)} \tilde{A}^{(1)(2)}. \quad (4)$$

$$\prod_{i=1}^{n-1} \tilde{A}^{(i)(i+1)} \leftarrow \tilde{A}^{(1)(2)} \tilde{A}^{(2)(3)} \dots \tilde{A}^{(n-1)(n)}. \quad (5)$$

In the equations above, n denotes the top structural layer, and the $\tilde{A}^{(i)(i+1)}$'s are normalized adjacency matrices channeling the i -th and the $(i+1)$ -th structural layers. $\tilde{A}^{(n)}$ symbolizes the message exchange process that occurs within the top layer

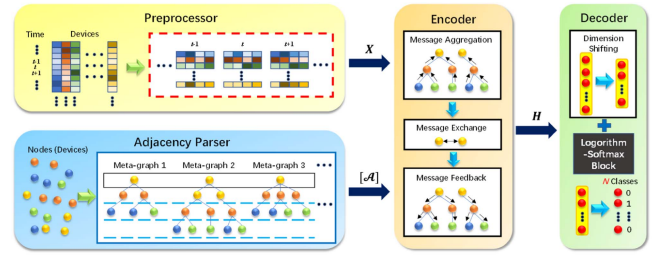


Fig. 5. HMA Architecture.

before message feedback. Details of these matrices are discussed in Section IV-B.

IV. MODEL STRUCTURE

The HMA encapsulates 4 primary building blocks, namely the Preprocessor, the Adjacency Parser, the Encoder and the Decoder. Its general architecture is illustrated in Fig. 5. Designed to deal with heterogeneous data sequences and complex relationships, the Preprocessor converts the sequential data input (series of sensor readings, actuator states, etc.) into vectors (initial messages) containing temporal characteristics of the original data at specific time ticks, and the Adjacency Parser parses the hierarchical relationships among all nodes of interest, producing the layered architecture on which subsequent tasks are performed. Their outputs are then passed to the Encoder, the core functioning component of the HMA which conducts meta-graph message aggregation, exchange and feedback as introduced in Section III-B. Finally, the updated messages are mapped to their states of inference in the Decoder. The main idea behind this design is to alter the amount of knowledge each node learns through message passing. By applying the hierarchical message aggregation scheme proposed in the Encoder on the layered architecture produced by the Adjacency Parser, compositions of learnt node embeddings are much less biased to the nodes' locality and are hence more balanced and reliable as a representation of the sophisticated relational semantics between a node and its surroundings.

A. Preprocessor

The input of the HMA is a stream Φ of M device readings obtained within a time interval of length T , represented as a matrix of dimension $T \times M$. For each individual stream φ (Φ 's column) in Φ with respect to a particular device, the Preprocessor transforms φ into a series of initial messages (vectors) of dimension γ via temporal encoding, a process that extracts temporal features within a time interval (window) of a preconfigured size placed at a specific time tick. With n the length of the window applied, each φ is converted into $(T - n + 1)$ messages of length γ , and thus the final output of the Preprocessor (denoted X) is of dimension $(T - n + 1) \times \gamma \times M$.

In order to extract the numeric features of a reading sequence at a given time tick t , temporal encoding is achieved via the Grouped Exponential Weighted Average (GEWA) algorithm (see Algorithm 1 and Fig. 6). This algorithm compresses all values within a window of length n into a new vector $x(t)$ of

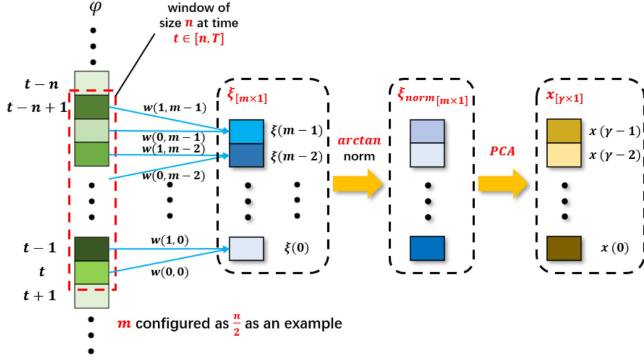


Fig. 6. The GEWA workflow (m set to $n/2$ as an example).

Algorithm 1: Grouped Exponential Weighted Average (GEWA).

Input: Reading streams of M devices $\Phi_{[T \times M]}$
Output: Preprocessed vectors $X_{[(T-n+1) \times \gamma \times M]}$
Ensure:

- 1: $X \leftarrow \text{emptylist}$
- 2: Initialize window size n and downsizing parameter m .
- 3: Compute exponential weights w (See (8))
- 4: **for** each device reading φ in Φ :
- 5: Vector list for current device $\Xi \leftarrow \text{emptylist}$
- 6: **for** $t \in [n, T]$:
- 7: Fetch window at t :
- 8: Compute $\xi(t)$ (See (6) & (7))
- 9: Normalize $\xi(t)$: $\xi_{\text{norm}}(t) \leftarrow \arctan(\xi(t))$
- 10: $\Xi \leftarrow \Xi + \xi_{\text{norm}}(t)$
- 11: **end for**
- 12: $X \leftarrow X + \Xi$ (Ξ is of size $[(T-n+1) \times m]$)
- 13: **end for**
- 14: Perform *PCA* over all m -dimensional vectors in X
- 15: **return** X

a downsized dimension γ , where $n \leq t \leq T$ is the time tick at which the window is captured. This $x(t)$ contains the reading's temporal properties at time t and can hence act as a node's initial feature vector representation used for succeeding tasks of message passing and state inference (at time t).

In Algorithm 1, the window of size n is evenly divided into m segments within each of which weighted averaging is conducted. This operation shrinks the n -dimensional window into a vector $\xi(t)$ of length m (See (6) and (7)).

$$\xi(t) = [\xi(t, 0), \xi(t, 1), \dots, \xi(t, m-1)]^T. \quad (6)$$

$$\xi(t, j) = \sum_{i=0}^{\frac{n}{m}} w(i, j) \cdot \varphi \left(t - i - \frac{jn}{m} \right) \quad (7)$$

Note that in (7), $t \geq n$, $0 \leq j < m$ and $w(i, j)$ regulates the contribution of the original entry recorded at time $(t - i - \frac{jn}{m})$ to $\xi(t, j)$. Assuming that the importance of certain context decays with time, the weighing factors $w(i, j)$'s in our implementation is hence set to decrease as time backtracks. In particular, we employ a set of exponential weights in this step of our preprocessing

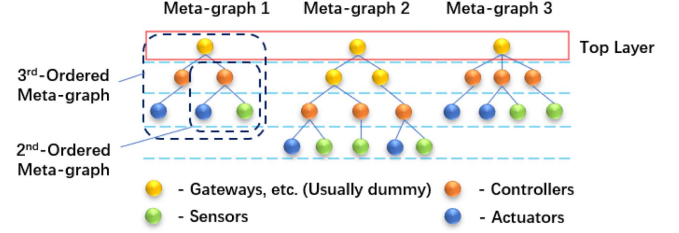


Fig. 7. Architecture Construction (Different Meta-graphs may correspond to distinct stages of an industrial process, and may vary among different industrial scenarios.).

procedure (See (8)).

$$w(i, j) = \frac{e^{-\frac{a}{n} \left(i + \frac{jn}{m} \right)}}{\sum_{j=0}^{m-1} \sum_{i=0}^{\frac{n}{m}-1} e^{-\frac{a}{n} \left(i + \frac{jn}{m} \right)}}. \quad (8)$$

where a is the coefficient that balances the distribution of all weights in a specific window.

Owing to the scaling distinctiveness of the original readings, additional normalization is applied to $\xi(t)$ after the averaging process is complete, to circumvent potential training bias toward the nodes with dominant magnitude. In our implementation, a simple *arctangent* normalization scheme is employed for the convenience that no additional statistics is required during the normalization process (See (9)).

$$\xi_{\text{norm}}(t, j) = \arctan(\xi(t, j)), \quad 0 \leq j < m. \quad (9)$$

Finally, the normalized vectors $\xi_{\text{norm}}(t)$'s are condensed to $x(t)$'s of dimension γ via Principal Component Analysis (PCA) for its meliorating effect on the model's performance (See (10)).

$$x(t) \leftarrow \text{PCA}(\xi_{\text{norm}}(t)). \quad (10)$$

Given that the $x(t)$ generated in (10) is associated with a particular device at a specific time tick t , the final output of the Preprocessor X is defined as an aggregation of the $x(t)$'s obtained at all $t \in [n, T]$ and for all φ 's in Φ .

In summary, by performing temporal compression, we transform the original reading sequences Φ into their respective graph node features at different time ticks, generating initial representations X for subsequent training.

B. Adjacency Parser

In general, the Adjacency Parser determines the hierarchical architecture for subsequent message passing via meta-graph alignment and layer assignment. As defined in Section III-B, each meta-graph has a supervisory node that collects data from other member nodes in the graph before transmitting it to its own supervisor in a higher-order meta-graph (See Fig. 7). For example, in a 2nd-ordered meta-graph $G_m^{(2)}$ composed of a single controller C and its subordinate sensors S 's and actuators A 's, the controller is the supervisory node dedicated to collecting and aggregating information from its immediate subsidiaries (S 's and A 's) and reporting to its own supervisory node in a 3rd-ordered meta-graph, which can be a security gateway, or a switch, etc. Fig. 8 exemplifies a 6th-ordered meta-graph for a secure water treatment process (SWaT) constructed with layers

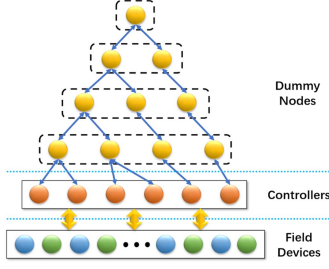


Fig. 8. 6-Layer Structure for SWaT Process.

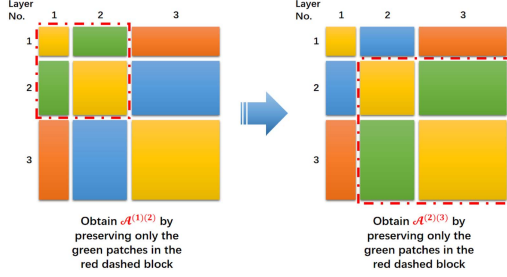


Fig. 9. Example of Structural Layer Dissolution.

of multiple field devices, controllers and upper supervisory nodes (denoted as dummy).

It is trivial to realize that every node in a graph can be treated as a supervisory node in a meta-graph of some order. Nonetheless, not all nodes are committed to both the tasks of data aggregation and reporting. In a typical ICS layout, field devices such as sensors and actuators usually do not have any subordinate devices connected to them, and hence their only task is to report to their supervisors, namely the controllers. Some other nodes, on the other hand, may not have any supervisors by configuration. For instance, in scenarios where no devices are taken into consideration other than the controllers and field devices, the controllers have no supervisors to report to. Thus, such nodes only receive messages during the data aggregation process, and they are aligned in the top layer of the hierarchical architecture. Once the top layer is generated, determining the rest of the structure is recursive and straightforward. Specifically, the nodes directly reporting to the top layer are assigned to the layer right below it (See Fig. 7).

In order to develop the intended hierarchical message passing function, we store the connectivity information among nodes corresponding to all pairs of adjacent structural layers in an array of adjacency matrices $[\mathcal{A}^{(i)(i+1)}]$ as defined in Section III-B. Each $\mathcal{A}^{(i)(i+1)}$ only inherits the associativity between layer i and $i+1$ and does not convey any internal links within one exclusive end, as the HMA only prompts internal message exchange on the top layer. Consider the adjacency matrix $\mathcal{A}$ of a 3-layered structure in Fig. 9, in which nodes that pertain to the same layer are grouped together so that the adjacency information between any pair of structural layers is conveniently decoupled into blocks (sub-matrices). One can clearly observe that the three yellow patches preserve the internal links within the same layers, while the blue and green patches store all the connections between neighbouring layers. In the left matrix in Fig. 9, specifically,

Algorithm 2: Adjacency Parser.

Input: Set V of M nodes of interest (controllers, field devices, etc.)

Output: List of adjacency matrices $\mathcal{A}$

Ensure:

- 1: $\mathcal{A} \leftarrow \text{emptylist}$, layers $\epsilon \leftarrow \text{emptylist}$
 - 2: Construct list G_m of meta-graphs using all nodes in V (See Fig. 7)
 - 3: $s \leftarrow \{\text{Supervisory nodes of all meta-graphs in } G_m\}$
 - 4: $\epsilon \leftarrow \epsilon + s$
 - 5: $V \leftarrow V - \{\text{Nodes in } s\}$
 - 6: **while** $V \neq \emptyset$:
 - 7: $r \leftarrow \{\text{Nodes in } V \text{ directly linked to nodes in } s\}$
 - 8: $s \leftarrow r$
 - 9: $\epsilon \leftarrow \epsilon + s$
 - 10: $V \leftarrow V - \{\text{Nodes in } s\}$
 - 11: **end while**
 - 12: Derive layer adjacency matrix $\mathcal{A}$ using layers in ϵ
 - 13: **for each** neighbouring pair of layers (α^i, α^{i+1}) in ϵ :
 - 14: Construct $\mathcal{A}^{(i)(i+1)}$ from $\mathcal{A}$ w.r.t. (α^i, α^{i+1}) via block masking.
 - 15: $\mathcal{A} \leftarrow \mathcal{A} + \mathcal{A}^{(i)(i+1)}$
 - 16: **end for**
 - 17: **return** $\mathcal{A}$
-

the green blocks in the red dashed box denotes all existing links between layer 1 and 2. Hence, $\mathcal{A}^{(1)(2)}$ can be derived by applying a mask to $\mathcal{A}$ that preserves only the green patches and sets the rest to zero. Similarly, the green patches in the right matrix in Fig. 9 renders the connection patterns between layer 2 and 3, and thus, we are able to obtain $\mathcal{A}^{(2)(3)}$ using the same masking principle.

To sum up, the workflow of the Adjacency Parser is presented in Algorithm 2. This module stipulates the layered structure and parses all the adjacency information between neighbouring structural layers in an array of matrices $[\mathcal{A}^{(i)(i+1)}]$, which serves as the 2nd part of input to the GNN-based Encoder.

Note that one of the core properties of an ICS process is long-term periodicity. It means that such a process is typically required to run for days, weeks or even months. To meet this need, an ICS system is usually a fixed structure whose set of devices and connections remain stationary for a long period of time. Thus in principle, the hierarchical architecture built above should exhibit little dynamic characteristics. However, in case of exceptions such as node addition or removal, changes occur in the constructed hierarchical architecture. In this case, Algorithm 2 should be timely executed again to update the elements in the array $\mathcal{A}$ of adjacency matrices, and subsequent training should be performed for the HMA model to adapt to this new architecture. Furthermore, if relations among devices are unavailable or irregular, one way to deal with this situation is to equally treat all devices by randomly arranging them into groups and assigning an additional dummy node in each group as its supervisory node. This creates hierarchical structures shown in Fig. 8, in which relations among the controllers are

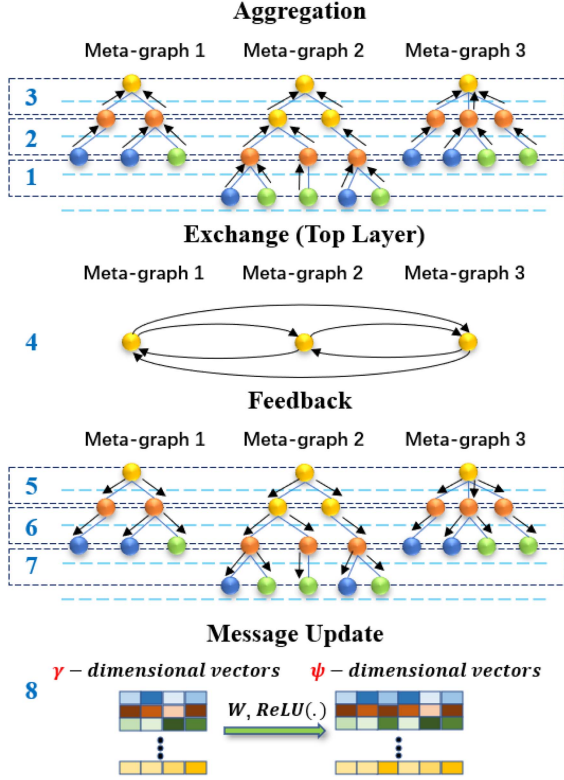


Fig. 10. Encoding Process (Each round of message passing is completed by sequentially following steps 1 through 8).

unknown and all controllers are connected via upper layer yellow dummy nodes. The dummy nodes here act as virtual message aggregators rather than real entities, and they enable construction of hierarchical architectures with no a priori information on devices' supervisory relationships.

C. Encoder

As the core function of the HMA, the Encoder enables hierarchical message passing via a cascade of message aggregation, exchange and feedback, followed by an $UPDATE(\cdot)$ operation for dimension shifting. The input to this module consists of the initial messages X of size $(T - n + 1) \times \gamma \times M$ produced by the Preprocessor, as well as the array of adjacency matrices $[\mathcal{A}^{(i)(i+1)}]$ manifesting the structure of the layered topology defined in the Adjacency Parser. The output of the Encoder is the updated messages H of dimension $(T - n + 1) \times \psi \times M$, where ψ equals the length of every updated message vector. Details of this encoding process is presented in Algorithm 3 and Fig. 10.

In Algorithm 3, initial messages (X) recursively ascend to the top via message aggregation, mingle at the peak level and descend back down to the bottom of the layered architecture as $\hat{H}$. Each ascent or descent is achieved via a left multiplication of a normalized adjacency matrix $\tilde{\mathcal{A}}^{(i)(i+1)}$, which acts as a bridge for the neighbouring layers i and $i + 1$ to transfer messages to each other. The $\hat{H}$ is subsequently updated to H by applying a linear weight matrix W and a $ReLU(\cdot)$ activation, and the generated H has a new dimension of $(T - n + 1) \times \psi \times M$.

Algorithm 3: Message Passing.

Input: Initial messages $X_{[(T-n+1) \times \gamma \times M]}$, List of adjacency matrices $\mathcal{A}$
Output: Updated message $H_{[(T-n+1) \times \psi \times M]}$
Ensure:

- 1: $H \leftarrow \text{emptylist}$
- 2: **for** $\mathcal{A}^{(i)(i+1)} \in \mathcal{A}$:
- 3: Add *self-loop* to $\mathcal{A}^{(i)(i+1)}$
- 4: $\mathcal{A}^{(i)(i+1)} \leftarrow \text{LaplacianNormalized}(\mathcal{A}^{(i)(i+1)})$
- 5: **end for**
- 6: **for** $t \in [n, T]$:
- 7: $\hat{H}(t) \leftarrow$ Matrix containing all M γ -dimensional vectors in X w.r.t time t
- 8: **for** $\mathcal{A}^{(i)(i+1)} \in \text{reversed}(\mathcal{A})$:
- 9: Message aggregation $\hat{H}(t) \leftarrow \mathcal{A}^{(i)(i+1)} \times \hat{H}(t)$
- 10: **end for**
- 11: Perform top layer message exchange.
- 12: **for** $\mathcal{A}^{(i)(i+1)} \in \mathcal{A}$:
- 13: Message feedback $\hat{H}(t) \leftarrow \mathcal{A}^{(i)(i+1)} \times \hat{H}(t)$
- 14: **end for**
- 15: $H(t) \leftarrow \hat{H}(t) \times W$
- 16: $H(t) \leftarrow \text{ReLU}(H(t))$
- 17: $H \leftarrow H + H(t)$
- 18: **end for**
- 19: **return** H

Note that in our implementation, Laplacian normalization is employed in each step of message passing due to its outstanding runtime efficiency compared against other mechanisms such as attention and sampling.

D. Decoder

To determine the states of all nodes in every time tick of interest, the updated messages H are fed into a decoding block with a Logarithm-Softmax layer and a category mapping function. The Logarithm-Softmax function converts each vector h in H , which is of length ψ , down to an N -dimensional vector of logits $\hat{y}$ where N denotes the number of classes. Then it computes the logarithmic probability $lp = [lp(1), lp(2), \dots, lp(N)]$ of all the N classes that a particular $\hat{y}$ is associated with in the following manner (See (11)).

$$lp(i) \leftarrow \log \left(\frac{e^{\hat{y}_i}}{\sum_{j=1}^N e^{\hat{y}_j}} \right), \quad 1 \leq i \leq N. \quad (11)$$

where $\hat{y}_i$ refers to the i -th entry in $\hat{y}$. Finally, all lp vectors are transformed into one-hot representations showing the categories these vectors are mapped to. The eventual output Y of the Decoder and the entire HMA is therefore of size $(T - n + 1) \times N \times M$, inferring the states of all M devices for all $(T - n + 1)$ time ticks.

As a binary output is generated for each node, operators may interpret the result based on the specific type of device being examined. In case of sensor, a positive output indicates a deviation in its measurement values from normal variational patterns, while for an actuator, this means that the device's

TABLE II
DATASET CHARACTERISTICS

Dataset	#Controllers	#Devices	Attack Duration	Sampling Interval	#Timestamps	#Node-level Samples	Anomaly Ratio
SWaT	6	51	4 days	1s	449,920	1,826,304	12.1402%
CISS	6	28	28 hours (within a 4-day period)	1s	115,883	630,784	0.0621%
WADI	5	123	2 days	1s	172,800	1,324,672	5.7737%
BATADAL	9	31	6 months	1hr	4,177	89,280	5.2430%
CHD	N/A	12	N/A	N/A	16,569,475	1,987,584	14.0712%

operating states are not properly switching as configured. For a controller, however, interpretation of an anomalous flag relies on the variable under check, which could be an adversarial command or an invalid operand, etc.

V. EVALUATION

In this section, we discuss our evaluation process including the used datasets, baseline configurations, relevant results and analysis. All baseline methods are measured on a server running CentOS Linux system (version 7) and equipped with 32 processors of 800.000 MHz together with a Memory of 125.388 G. All necessary coding is implemented using PyTorch version 1.7.1+cu110. Popular metrics (F1 Score, AUROC and AUPRC) are adopted in the performance assessment process. As described in Section IV, the HMA's input and output sizes are $T \times M$ and $(T - n + 1) \times N \times M$, respectively. With T , N and M the number of time ticks, classes and devices, the output contains the anomalous states of every individual device for all time ticks of interest. As we do not distinguish anomalies of disparate categories in this study, all final output states are binary (i.e., $N = 2$, output state equals 0 for normal and 1 otherwise).

A. Datasets & Graphs

We measure the HMA's functionality over 5 popular datasets in security research: the Secure Water Treatment dataset (SWaT) [55], the Critical Infrastructure Security Showdown dataset (CISS), the Water Distribution dataset (WADI: A water distribution testbed for research in the design of secure cyber physical systems), the Battle of Attack Detection Algorithms dataset (BATADAL) [56] and the Car Hacking Dataset (CHD). All datasets are ICS-based except the CHD, which is a car hacking dataset recording a vehicle's CAN traffic in 4 attack scenarios. This dataset is used to evaluate the HMA's generality in domains other than ICS security. The key characteristics of all 5 datasets are summarized in Table II. Note that with the system-level operating states provided in the last column of each data set, device-wise labels are generated from these states in order to perform fine-grained node-level anomaly detection. Specifically, a snippet extracted from a device's data stream is labelled as anomalous if any state within the snippet is anomalous in the original dataset.

The ICS network topologies on which model evaluation is conducted are exemplified in Fig. 11. In both architectures, all field devices (sensors and actuators) are directly linked to their respective controllers, while all controllers are inter-connected in a 2-layered architecture but associated with a dummy supervisory node denoted as "Central Reference Point" (or "CRP") in a 3-layered instance.

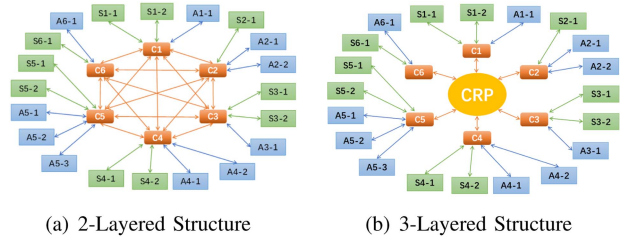


Fig. 11. Layered Structures.

TABLE III
CORE HMA PARAMETERS

Dataset	Preprocessor	Adjacency Parser	Encoder	Decoder
SWaT	$n = 4300, m = 43, a = 4$	$M = 6 + 35$	$\psi = 128$	$N = 2$
CISS	$n = 3000, m = 30, a = 4$	$M = 6 + 50$	$\psi = 128$	$N = 2$
WADI	$n = 5000, m = 50, a = 4$	$M = 5 + 74$	$\psi = 128$	$N = 2$
BATADAL	$n = 100, m = 10, a = 4$	$M = 9 + 36$	$\psi = 128$	$N = 2$
CHD	$n = 100, m = 10, a = 4$	$M = 3 + 9$	$\psi = 128$	$N = 2$

the HMA's performance, we implement the architectures with 4, 5 and 6 structural layers as well. Relevant test results are shown in the following sections. Note that in the datasets above, there are devices whose readings remain constant all along. These devices are omitted in the graph assuming that they are not affected by the physical processes (e.g. a shutdown pump whose state remains zero).

B. Baselines

The HMA's implementation complies with the configurations in Table III. Note that the M values are quantitative additions of controllers and field device readings ($M = 6 + 35$ stands for 6 controllers plus 35 readings from the field devices). They do not include any dummy nodes involved in architectures with deep structural layers.

Our set of baselines consists of several GNN-based benchmark approaches including classic GNN variations (GAT, TAGCN) and more advanced methods recently proposed for adversary detection (GLIN, AHGA, etc.), as well as several popular methods for outlier detection (GAAL, LOF, AE, etc.). All benchmark methods and their implementation details are listed as follows:

- 1) *GAT* [44]: A GNN framework with an attention mechanism. Input layer size equals the length γ of preprocessed vectors, and the output layer is of size 2. Between the input and output layers, 1 hidden layer with 128 neurons is configured.
- 2) *TAGCN* [20]: Compound GNN with a kernel that consists of multiple classic Graph Neural Network channels. The number of neurons in the input layer matches the

TABLE IV
BASELINE TEST RESULTS

Methods	#Structural Layers	SWaT			WADI			BATADAL			CISS			CHD		
		F1	AUROC	AUPRC	F1	AUROC	AUPRC	F1	AUROC	AUPRC	F1	AUROC	AUPRC	F1	AUROC	AUPRC
HMA	2	82.93	87.03	81.63	91.24	96.95	93.96	79.85	85.36	69.44	82.21	87.13	89.17	99.18	99.63	99.54
	3	81.63	86.20	79.59	88.58	94.33	90.97	78.46	83.04	65.06	80.14	84.15	86.02	99.33	99.63	99.50
GAT	N/A	70.95	73.75	59.03	59.92	69.85	49.13	68.24	66.24	33.94	70.35	68.09	70.36	87.55	94.86	90.38
TAGCN	N/A	68.88	69.86	52.30	62.57	64.99	45.79	65.78	64.65	24.79	73.05	67.91	69.12	71.39	89.39	80.06
GLIN	N/A	79.51	85.99	78.64	85.64	94.29	89.16	80.39	82.03	64.12	73.38	78.06	80.85	98.61	99.63	99.48
AHGA	N/A	78.16	82.75	73.83	87.86	93.01	84.99	65.20	70.39	38.44	71.87	77.45	79.67	86.48	94.85	89.79
TPGraph	N/A	73.17	77.18	64.91	67.77	77.78	61.20	61.07	70.64	31.47	63.31	68.33	76.01	96.66	99.37	98.87
GSLCN-LR	N/A	73.17	75.19	62.25	66.85	70.47	55.23	61.61	68.02	29.86	70.37	71.32	76.39	85.69	94.16	89.32
GSLCN-SR	N/A	73.11	75.03	62.46	70.14	75.85	61.82	63.25	69.91	31.81	72.36	70.48	75.38	85.41	93.99	89.12
GAAL	N/A	52.72	58.50	30.64	61.55	49.79	30.50	63.13	50.26	49.07	59.93	65.36	68.43	65.34	70.89	40.57
LOF	N/A	57.39	60.10	33.50	53.58	55.90	33.32	53.75	51.64	19.65	55.75	56.04	67.76	67.65	76.16	52.97
AE	N/A	51.56	57.69	30.65	53.25	56.39	34.24	54.86	62.02	17.82	40.28	43.79	59.26	60.18	74.37	54.02
OCSVM	N/A	53.22	46.16	27.39	53.55	51.71	32.92	53.37	42.89	13.12	57.21	49.37	61.08	60.05	51.06	40.40
Isolation Forest	N/A	53.83	58.35	32.50	53.24	56.13	33.05	54.20	64.17	20.85	59.84	62.49	67.11	62.00	47.87	40.24

dimension γ of the preprocessed vectors. Quantity of neurons in the output layer equals 2. Maximum of 3 hidden layers of size 128 are implemented.

- 3) *GLIN* [51]: A global pooling and integration approach applied to the decoding block of a GNN framework. It has an input layer with γ neurons, 2 hidden layers with 128 neurons, and an output layer of size 2. Maximum pooling and concatenation are applied.
- 4) *AHGA* [57]: Features graph-learning using process-related properties. Implemented with an input layer of size γ , 3 hidden layers of size 128 and a binary output layer.
- 5) *TPGraph* [42]: Graph learning scheme using feature correlation mining. Specifically developed with 2 graph producing blocks (one for $\hat{A}_r$ and one for $\hat{A}_c$) and 1 graph fusion module. Anomaly detector is constructed as a GCN structure with 2 hidden layers of 64 neurons.
- 6) *GSLCN* [41]: Graph learning approach based on long-term or short-term dependency encoding. Coordinates replaced by an ICS device's position attributes and features computed as a vector with 64 entries, mapped from original numeric data streams.
- 7) *GAAL* [58]: Features application of informative sample generation and boundary learning in anomaly detection. MO-GAAL is developed with 2 generators, each of which composed of 3 layers, as well as 1 discriminator equipped with 1 input layer of size γ , 1 hidden layer of size 128 and 1 output layer of size 2.
- 8) *LOF* [59]: Density-based outlier detection scheme on assumption that regular nodes are clustered more closely than outliers. Its *contamination* rate equals the rate of anomalies in each preprocessed dataset.
- 9) *AE* [60]: Anomaly detection method based upon embedding reconstruction. Implemented as a 3-layered architecture with 1 input layer of size γ , 1 encoding layer of size 2^k where k equals the largest integer that satisfies $2^k \leq \gamma$, and 1 decoding layer of the same size as the input.
- 10) *OCSVM* [61]: An unsupervised outlier detector based upon hyperplane training. Similar to the Isolation Forest, the *nu* rate is set to the rate of anomalous samples in the preprocessed data corresponding to each dataset.
- 11) *Isolation Forest* [62]: An unsupervised outlier detector using feature point splitting. Its *contamination* rate

matches the anomalous rate in the set of preprocessed samples with respect to each dataset.

Note that the output vectors generated by the Preprocessor in Section IV-A serve as the input of all the baseline methods.

C. Baseline Test Results

Comparative test set results are summarized in Table IV. For the proposed HMA method, metric values of the models trained on hierarchical graphs with 2 and 3 structural layers are recorded for comparison. Note that while the depth of the hierarchical architecture can be greater than 3, these scenarios are analyzed separately in Section V-D3.

As shown in Table IV, it is clear that the HMA's detection performance surpasses the benchmark methods in almost all metrics for all the 5 datasets. Compared with its GNN counterparts, it achieves average AUROC gains of 9.92%, 18.92%, 15.09%, 15.47% and 4.45% with respect to SWaT, WADI, BATADAL, CISS and CHD datasets. As all the baseline approaches follow the conventional neighbouring aggregation paradigm during the message passing process, it can be inferred that hierarchical message aggregation may have a more extensive meliorating effect on the quality of generated node embeddings, with which anomaly detection is conducted. With contribution of each node to a particular vector representation reshaped, the message composition pattern of every node embedding is a better reflection of how a node is hierarchically associated with the rest of the topology, such as whether specific nodes are positioned in the same layer of an ICS architecture, etc. The resulting node embeddings are, therefore, semantically closer to the specified ICS structure and serve as a more informative and reliable source for subsequent node-level anomaly detection.

One may also claim from Table IV that the HMA outperforms non-GNN-based methods including the GAAL, LOF, AE, etc. In terms of node-level detection, the feature vectors used by these methods do not contain information from other devices, and therefore cannot utilize the correlations among nodes to enhance adversarial numeric patterns for anomaly detection. In SWaT scenario, for example, the vector representing FIT-101 (measuring flows) incorporates features of sensor FIT-101 only, but does not gather characteristics of its neighbours, say, sensor LIT-101 (measuring liquid levels in tanks). As the containers (tanks or pipes) are connected in which LIT-101 and FIT-101 are deployed, variation in one's numeric streams presumably has

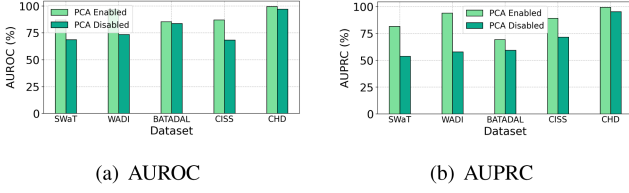


Fig. 12. Test results with and without PCA.

a direct influence on the other (Level changes in tanks lead to flow changes in pipes). With LIT-101's information encoded, the FIT-101's feature vector should gain a better perception of how its own measuring value changes, and become more sensitive to an anomaly at its occurrence. However, such neighbouring interactions do not exist in these methods, and as a consequence, the methods' reliability in anomaly detection is undesirably limited.

D. Ablation Study

Apart from the baseline test results provided above, a comprehensive ablation study is conducted in order to explore how each component of the HMA framework shapes the model's performance. Effects of core functions in all components are measured, except for the Decoder whose structural configurations remain consistent across all HMA variants. In particular, the following items are examined: the PCA function in the Preprocessor, the hierarchical dissolution function and structural layer configuration in the Adjacency Parser, as well as the 3 primary hierarchical message passing steps (message aggregation, top layer message exchange, message feedback) in the Encoder.

1) *PCA Function*: Comparative test results of the HMAs implemented with and without the PCA function in the Preprocessor are presented in Fig. 12. The noticeable numeric discrepancies between the 2 scenarios suggest that the HMA's performance can be rather susceptible to the quality of initial input features. Core essences extracted from the original data streams, redundant information that negatively impact the HMA's learning process is mostly removed via PCA, leading to a boost in the quality of the model's input features. The performance of the HMA trained with these refined attributes tends to be greatly enhanced as a result.

2) *Hierarchical Dissolution*: The hierarchical dissolution function dissects the ICS topology into layers, based on which data transmission paths are built. In order to examine its effect on the HMA's detection ability, the original HMA is constructed, together with the classic GCN trained with the same layered graph input but regular neighbouring aggregation scheme. Results yielded by these two implementations, as exhibited in Fig. 13, are an immediate demonstration of the superiority of the hierarchical message passing paradigm over the classic neighbouring aggregation convention. With data transmission paths re-regulated across distinct levels, core layer-wise network semantics are better absorbed in the nodes' vector representations, improving the model's adaptation to the respective ICS systems and enhancing its anomaly detection ability.

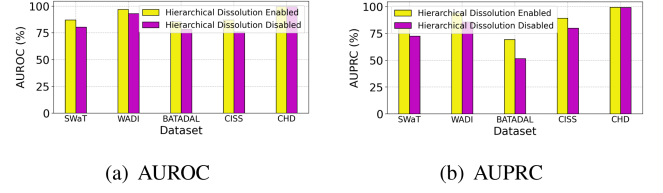


Fig. 13. Test results with and without hierarchical dissolution.

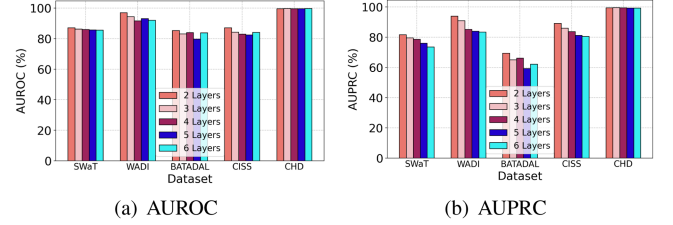


Fig. 14. Test results for different structural layer configurations.

3) *Structural Layer Configuration*: As is demonstrated in Section V-C, results produced by the HMA implemented with a 2 or 3 layered hierarchical architecture appear rather decent. This gives rise to the conjecture, though, as to whether the HMA's performance can be further optimized via introducing extra structural layers. To explore this, additional layers are stacked above the controllers to deepen the structure. These layers contain dummy nodes which do not have any specific physical roles. In a 6-layer structure constructed for the SWaT dataset (See Fig. 8), for instance, a layer of 4 nodes is placed above the controllers, among which 2 are each linked to 2 controllers, while the other 2 are uniquely connected to the remaining 2 controllers. Then another layer of 3 nodes is built upon the 4-node layer, and the mapping between these 2 layers are established in a similar way.

Assessment results on the HMA trained on graphs with 2 to 6 structural layers are illustrated in Fig. 14. For all datasets evaluated, performance is peaked on topologies with either 2 or 3 layers. In addition, as observed from the SWaT, WADI and BATADAL histograms, the HMA's functionality generally falls as more layers are involved. This possibly attributes to the over-smoothing effect caused by excessive message passing steps on deep structural layers.

4) *Hierarchical Message Passing*: Finally, experiments are conducted to explore how each step of the proposed hierarchical message passing scheme impacts the model's detection functionality. With the original scheme sequentially composed of message aggregation, exchange and feedback, scenarios are investigated in which parts of the steps are disabled. The examined scenarios and the corresponding test results are displayed in Table V. It can be easily discerned from the results that the metric values drop faster with more steps removed. The AUPRC for the SWaT dataset, for instance, suffers from a 2.33%, 13.53% and 24.24% decline when message feedback, exchange and aggregation are orderly blocked. On one hand, it is inferred that as the initial step of the whole message passing scheme, message aggregation plays a significant role in reshaping a node's awareness of its contextual correlations with the rest of the graph.

TABLE V
FUNCTIONAL IMPACT OF SPECIFIC HIERARCHICAL MESSAGE PASSING STEPS

Step Configurations	SWaT			WADI			BATADAL			CISS			CHD		
	F1	AUROC	AUPRC	F1	AUROC	AUPRC	F1	AUROC	AUPRC	F1	AUROC	AUPRC	F1	AUROC	AUPRC
Original	82.93	87.03	81.63	91.24	96.95	93.96	79.85	85.36	69.44	82.21	87.13	89.17	99.18	99.63	99.54
w/o message feedback	80.02	86.48	79.30	88.87	94.65	90.73	78.94	83.19	61.95	77.25	85.81	88.43	94.53	99.00	98.40
w/o message feedback or exchange	73.46	76.62	65.77	75.24	84.18	72.42	69.67	67.54	41.17	72.24	74.00	78.58	83.39	93.63	87.68
w/o message feedback or exchange or aggregation	64.28	60.85	41.53	62.16	60.25	42.93	62.28	63.24	23.35	69.05	65.95	70.78	67.53	71.99	69.99

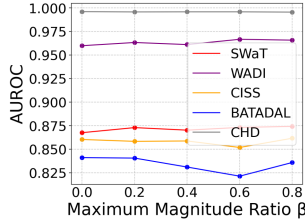


Fig. 15. Test results with noise.

On the other hand, even though not as prominent, the message exchange step also has a rather large positive influence on the HMA, as it is the main step in which information from nodes in disparate industrial stages gets mingled and integrated. With this operation, further enhancement in the nodes' perception of their surroundings is achieved, leading to a rather promising performance gain in the HMA's detection functionality.

E. Robustness Study

In this section, the HMA's resiliency is examined against multiple external interfering factors, including varying ratio of anomalies, as well as graph input errors such as structural perturbation and environmental noise. Implementation details and the respective test set results and discussions are given as follows.

1) *Influence of Noise Addition:* To examine the effect of noise in input graphs on the HMA's detection performance, a uniformly distributed noise is applied to every non-zero element in the equivalent adjacency matrix A_{eq} used for forward propagation. In particular, this equivalent adjacency matrix can be expressed as $(\prod_{i=1}^{n-1} \tilde{A}^{(i)(i+1)}) \tilde{A}^{(n)} (\prod_{i=1}^{n-1} \tilde{A}^{(n-i)(n+1-i)})$ where $\prod_{i=1}^{n-1} \tilde{A}^{(n-i)(n+1-i)}$ and $\prod_{i=1}^{n-1} \tilde{A}^{(i)(i+1)}$ are respectively defined in (4) and (5). With $A_{eq}^{\min}$ the minima of A_{eq} , each noise value is randomly generated within the range of $[-\frac{\beta}{2} A_{eq}^{\min}, \frac{\beta}{2} A_{eq}^{\min}]$, where β is a non-negative coefficient representing the maximum magnitude ratio between the noise and $A_{eq}^{\min}$. To investigate how noise of different magnitudes impacts the HMA's detection performance, multiple β values are selected for noise generation and the respective AUROC values of the HMA models are shown in Fig. 15.

Intuitively, as the maximum magnitude ratio increases, noise becomes stronger and tends to disturb the HMA more. However, as shown in Fig. 15, no apparent discrepancies are spotted among the AUROC values yielded in different β configurations for the majority of the datasets evaluated. This potentially attributes to the HMA's ability to smooth out the effects of noise via message aggregation. Therefore, the model's robustness is strengthened and its detection performance is hardly affected.

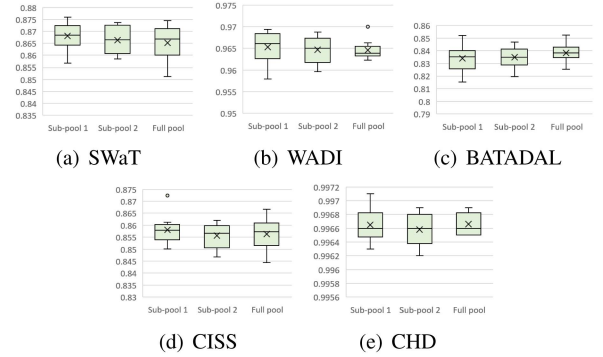


Fig. 16. Test results with structural errors.

2) *Influence of Structural Errors:* Apart from the environmental noise discussed above, structural errors are another factor that leads to distortion in input graphs and hence may negatively influence the HMA's detection accuracy. To explore this correlation, directed edges in the original graph are randomly dropped before construction of the hierarchical architecture for message passing. Specifically, in a 2-layered scenario shown in Fig. 11, edges are grouped into 2 sub-pools, i.e., sub-pool 1 containing all internal edges in the top layer and sub-pool 2 incorporating all edges between the 2 layers. A specified number (e.g., 5% of the total edge quantity) of edges are randomly selected and removed from designated pools for ten times and the respective AUROC values of the HMA models are displayed in the box plots in Fig. 16 (Note that the full pool in Fig. 16 denotes the entire edge set).

In general, the HMA's AUROC values fall within a rather narrow interval that approximately peaks at the AUROC obtained with the complete graph structure. This is the case for all 3 scenarios in which links are respectively deleted from sub-pool 1, sub-pool 2 and the entire edge set. It shows that disconnecting a limited number of node pairs does not lead to noticeable information loss for the rest of the graph. This information loss is actually compensated via alternative connections in the original graph. Hence, the HMA's robustness in terms of structural loss is justified.

3) *Influence of Anomaly Ratios:* As a key property of the input data, anomaly ratio also has a prospective influence on the HMA's functional reliability. To explore this connection, portions of anomalies are removed and the AUROC values of the HMA frameworks evaluated on the resulting datasets are presented in Fig. 17. Note that the horizontal indexes in Fig. 17 denote the ratio of removed anomalies to all anomalous samples in the original dataset.

One may tell from Fig. 17 that the HMA's detection performance remains stable around a particular level as the quantity

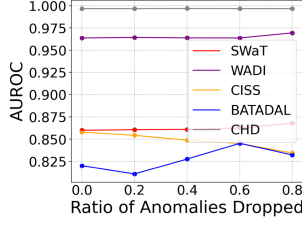


Fig. 17. Test results with different anomaly ratios.

of anomalous samples declines. It shows that the framework is able to capture sufficient variational semantics from the input streams that help distinguish anomalous patterns from the normal ones regardless of specific anomaly ratios. This contributes to the overall robustness of the proposed approach in anomaly detection.

F. Generalization Study

In this section, the HMA's generalization ability across non-ICS domains and its transferability to different scenarios are analyzed.

1) *Domain Generalization*: With the model's detection accuracy evaluated on not only popular ICS datasets, but also a car hacking dataset (CHD), it's convincing that the HMA is able to generalize in domains other than ICS security. Theoretically, whether the HMA can be applied to non-ICS scenarios depends on the feasibility of generating hierarchical structures from current graph topologies. Unlike an ICS system whose architecture is naturally layered and hierarchical, a non-ICS graph structure can be arbitrary and irregular. However, it is viable that these non-ICS graph structures get transformed into layered topologies using either of the following two strategies.

For heterogeneous graphs representing communities in which roles of all nodes are explicitly designated, hierarchical relations among nodes can be inferred from the specified roles. In a data collection scenario described in Section III-B, for instance, with the blue and green nodes referred to as regular members of the community, and the orange ones supervisors collecting data from groups of regular members, hierarchical relations (see Fig. 3(b)) can be established among the orange nodes and the union of blue and green ones. Similarly, links between the orange nodes and the yellow nodes can be created in the same fashion.

For all other cases in which either roles are unavailable or supervisory relations among roles are unclear, nodes are intuitively indistinguishable. However, as most of a node's interactions with the rest of the graph can still be considered bounded within a local distance, regions can be assumed and groups can be defined as sets of all nodes within the same region. In this case, communication among different regions requires construction of inter-region highways (or channels). Such channels incorporate information from nodes in the same region and can be considered as supervisors of particular regions. Hierarchical links are hence produced among these channels and the respective regions.

With supervisory relations inferred and layered architecture created, the HMA can be generalized beyond ICS contexts and is

TABLE VI
TEST SET AUROC RESULTS FOR MODELS EVALUATED OVER ALL DATASETS
(LEFTMOST COLUMN: TRAINING SETS, TOP ROW: TEST SETS)

Datasets	SWaT	WADI	BATADAL	CISS	CHD
SWaT	0.8699	0.6852	0.6860	0.5717	0.7130
WADI	0.4392	0.9844	0.6093	0.6005	0.5428
BATADAL	0.5924	0.6820	0.8407	0.6387	0.6341
CISS	0.6169	0.5234	0.6371	0.8621	0.4545
CHD	0.6012	0.5449	0.6039	0.6372	0.9962

applicable to non-ICS scenarios in which hierarchical properties are unclear.

2) *Cross-Dataset Transferability*: Despite its good generalizability across non-ICS domains, whether the proposed framework trained in one system can be directly transferred to another is a completely different story. In this section, the models trained in one dataset are evaluated with the test samples in all other 4 datasets. All AUROC values are summarized in Table VI.

The results in Table VI shows that the HMA framework trained over one dataset does not fit the test samples in other datasets very well. For instance, the AUROC of the model trained over the SWaT dataset and tested on the CHD dataset is 0.2832 inferior than the one specifically trained for the CHD dataset. It is likely that this discrepancy attributes to the distinctiveness of both features and graphs in different domains. Therefore, in order to bridge the gap, the following strategies can be investigated in the future.

Feature Adaptation: The training and test datasets belong to different domains (e.g., trained on SWaT and tested on WADI), and their input vector spaces may differ a lot from each other, leading to incompatibility of a framework trained over one domain with a different one. Therefore, feature adaptation can be investigated in the future to map the vector space of test domain to the respective training domain (e.g., vector shifting or matrix transformation can be adopted to transform the WADI input patterns to the SWaT ones at the start of the testing process). In this fashion, the model's adaptivity to different domains is enhanced and its transferability can potentially be improved.

Graph Adaptation: Graph distinctiveness is another factor that adds to the model's inapplicability to different application scenarios, as it leads to a difference in terms of how information is aggregated. One potential solution that might mitigate this undesirable effect is creating auxiliary topologies that structurally bridge the test set graph to the training one (e.g., sub-graphs from test set graph that resemble part of the training graph). Using these auxiliary topologies, test set message passing is performed in a graph context similar to the one producing the trained detector. This presumably leads to a boost in the model's transferability among distinctive graph settings.

G. Runtime and Memory Consumption Analysis

To investigate the feasibility of the HMA's application in real-time anomaly detection, runtime comparison is conducted with the classic GCN method as well as all baseline approaches in Section V-B. In addition, in order to measure the amount of

TABLE VII
TRAINING TIME (PER EPOCH) AND TEST TIME (PER TIMESTAMP) FOR THE HMA AND GCN

Runtime	SWaT		WADI		BATADAL		CISS		CHD	
Consumption	Training	Test	Training	Test	Training	Test	Training	Test	Training	Test
HMA	1.2224	0.0210	1.5925	0.0623	0.2101	0.0187	0.4851	0.0335	1.2164	0.0053
GCN	0.4488	0.0095	0.3629	0.0168	0.1937	0.0085	0.2701	0.0134	0.8662	0.0048

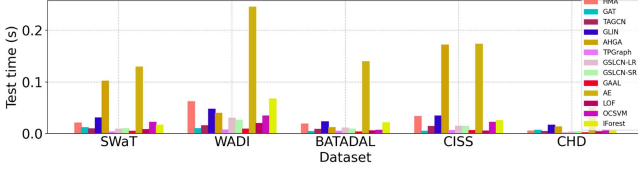


Fig. 18. Baseline Runtime Comparison.

spatial resource required for the operation of the HMA framework, its memory consumption is analyzed together with all the baselines.

1) *Runtime Comparison With Classic GCN*: Adopted structures and parameter configurations identical for both models, the training and test time of the HMA and GCN are presented in Table VII. Note that the training time is measured for each epoch and the test time is recorded on the basis of timestamp, indicating how long detection results are produced for all devices at a single sampling time.

One can discern that the HMA scheme suffers from an undesirable training and testing latency in contrast with the classic GCN method. This drawback arises from the increase of complexity in the adjacency matrices applied. Considering the SWaT topology with 3 structural layers, only 7,936 (0.11%) out of 7,225,344 entries are positive values in a GCN batch adjacency matrix incorporating 64 time ticks, while there are totally 112,896 non-zero entries (1.56%) in the respective HMA batch matrix. Similar findings are claimed in the 2-layered structures as well, with the number of existing edges 9,024 (0.13%) and 107,584 (1.56%) with respect to GCN and HMA. In terms of operating time complexity, as the main message aggregation steps in both methods are developed via sparse matrix manipulation, the methods' runtime consumptions are closely associated with the time complexity of the respective sparse matrix multiplication operations. Given 2 sparse matrices of shape $m \times n$ and $n \times p$, the time complexity of a regular matrix multiplication equals $O(mnp)$, but may drop down to as low as $O(mp)$ when sparse matrix operations are involved, in which case the specific time complexity varies proportionately with the ratio of non-zero elements in the adjacency matrices. As the HMA creates more non-zero values (existing edges) in its adjacency matrices, the level of matrix sparsity is lower and hence, the operating time complexity is relatively higher.

To alleviate this negative impact, the idea of reducing the number of nodes by clustering devices with similar reading patterns can be studied in the future. However, one should further investigate efficient ways to differentiate nodes in the same cluster to conduct node-level state inference.

2) *Runtime Comparison With Baselines*: Fig. 18 presents the test time results for the HMA and all the baseline methods. It shows that the HMA's test time consumption is generally on a

TABLE VIII
BASELINE MEMORY CONSUMPTION OF GNN-BASED METHODS

Mem Cost (KB)	HMA	GAT	TAGCN	GLIN	AHGA	TPGraph	GSLCN-LR	GSLCN-SR
Adjacency	19.70	1.65	1.45	1.45	10.01	19.48	4.35	5.77
Weight	132.00	4.00	132.00	68.00	196.00	68.00	68.00	68.00
Total	151.70	5.65	133.45	69.45	206.01	87.48	72.35	73.77

TABLE IX
BASELINE MEMORY CONSUMPTION OF NON-GNN-BASED METHODS

Mem Cost (KB)	GAAL	AE	LOF	IForest	OCSVM
Total	4.00	10.75	0.98	6.00	0.17

low level in contrast with some of the baseline approaches. With 0.0210 s, 0.0623 s, 0.0187 s, 0.0335 s and 0.0053 s the processing time of all device-wise inputs for a particular sampling time with respect to SWaT, WADI, BATADAL, CISS and CHD datasets, the method is suitable for real-time anomaly detection in these systems in which sampling interval is no less than 1 s. Furthermore, as these processing time values are approximately proportionate to the scale of the input architecture (i.e. number of nodes in the input graph), one may estimate that the HMA is capable of handling ICS systems (DCS, SCADA, etc.) with as many as over one thousand devices. Nevertheless, compared to over half of the GNN-based frameworks, this method's runtime efficiency still has room for improvement owing to the extra complexity in its adjacency matrices.

3) *Memory Consumption*: The memory costs of all GNN-based models are determined by the amount of space allocated for storage of both tuples of sparse adjacency matrices and parameters in weight matrices. Suppose coordinates are encoded as unsigned integers and all values (in tuples) and parameters are defined as float numbers, each element takes up 4 bytes of memory. Therefore, the total amount of memory consumed by each model can be estimated as $3 \sum_{i=1}^{n_{adj}} N_{tup}^i + \sum_{j=1}^{n_w} N_j$, where n_{adj} and n_w are quantities of adjacency matrices and weight matrices, and N_{tup}^i and N_j are numbers of tuples in the i -th adjacency matrix and values in the j -th weight matrix. The space costs of the HMA and all the GNN baseline implementations are shown in Table VIII.

For non-GNN-based benchmark approaches, how space costs are computed varies a lot from one method to another. For neural network based frameworks (GAAL and AE), memory usage depends on the size of parameter matrices between layers, and for outlier based methods (IForest, OCSVM and LOF), it is directly shaped by the quantity and dimension of samples used for construction of detectors. The estimated space costs of non-GNN baseline implementations are shown in Table IX.

Compared to most baseline methods, the HMA requires more memory for operation. However, in order to segregate the detection operation from industrial processes, the platform on which the framework runs is typically deployed in a separate branch from an aggregation node (e.g., a gateway). In this fashion, the HMA simply processes the data copied from this node on its local platform without consuming space resources of ICS devices. Typical configuration of such platforms differs a lot among systems and is customizable. Processors are usually of 1.8 to 3.5 GHz and available memory can be 16 G, 32 G or higher.

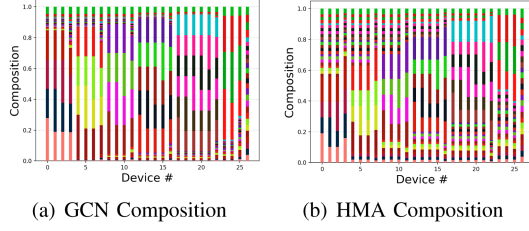


Fig. 19. Embedding Composition (Each bar represents a device embedding whose portions of different colours manifest the level of contribution of other devices).

For complex data processing tasks, superior configurations are also necessary.

H. Embedding Composition and Interpretability

The HMA establishes a new message passing scheme which enables the model to capture the vertical (hierarchical) relational semantics in typical ICS networks during the embedding learning process. Compared with the regular GCN, which indiscriminately treats every node regardless of their role in the system, the HMA intensifies a node's specificity via reformulating its embedding composition in a way that enhances its awareness of the roles played by other nodes' in the graph, as well as reducing any local discrimination through top-layer message exchange. It is apparent that different embedding compositions, which is determined by the disparate contribution patterns to a particular node from the rest of the graph, produce different inference results. Hence, taking the SWaT dataset as an example, we present the composition patterns for parts of the node embeddings produced by the HMA and GCN, as shown in Fig. 19.

One can observe from Fig. 19 that in both models, there are dominant contributions for each node embedding. Nevertheless, the distribution patterns in the HMA is dramatically more balanced than the GCN. Taking device #0 as an example (the leftmost column in each graph), its immediate neighbourhood (device #2, #3 and #4, the bottom 3 segments in the column) contributes approximately 50% to its representation produced by an HMA model, while over 75% in the case of GCN. This indicates that an overall balanced message layout may have a positive effect on the model's classification ability, on the premises that this layout is still far from being uniform, which leads to over-smoothing. The underlying distribution principle still requires further research, though.

To further interpret how the ultimate embedding composition for a specific node is generated, device #0 (FIT-101) is selected as an example whose composition patterns after 1, 2 and 4 rounds of GCN message passing are shown in Fig. 20. Each bar in the figure profiles the scale of contribution of each node to FIT-101's embedding. It can be inferred that after the 1st round of message passing, FIT-101's vector representation incorporates information from device #4 (the controller FIT-101 is directly linked to) other than itself, as it is the only node directly linked to FIT-101 in the graph. With the 2nd round completed, node FIT-101 is semantically enriched with all other



Fig. 20. Embedding composition for device #0.

nodes in the same industrial stage including device #2 (LIT-101), device #3 (MV-101), etc. As the number of rounds increases, device #0 starts incorporating information from nodes in other stages. From the composition patterns obtained after 4 rounds of message passing, it can be inferred that device #0 has gathered information from all other nodes in the graph, with nodes in the same industrial stage its dominant information sources. Embedding composition patterns for the HMA scenario are computed in the same manner.

Note that in both compositions presented in Fig. 19, the dummy node's contribution, denoted as the top green segment in every column, seems fairly prominent as well, even though this node does not have any direct role to play in the physical processes. This results from the principle of the inherent message passing mechanisms employed in the two models. For GCN, the further a node is, the less it contributes to the construction of embedding vectors. Similarly, in HMA, the significance of some node is diminished along the message aggregation and feedback paths. Since the dummy node information is encoded at the end of the message aggregation step, its contribution is fairly significant at this current point and only starts to shrink in the message feedback step. Therefore, at the end of message passing, this node still has noticeable influence on the embeddings.

VI. CONCLUSION

In this article, we investigate a hierarchical message passing scheme to generate effective embeddings for downstream anomaly detection tasks in the ICSs. We design and develop the HMA, a new message passing approach that re-regulates the message transmission directions via meta-graph data aggregation. The HMA is composed of 4 functional blocks known as the Preprocessor, the Adjacency Parser, the Encoder and the Decoder. It improves the anomaly detection accuracy by creating more balanced and reliable node representations encapsulating the sophisticated supervisory semantics in the ICSs for state inference. The framework's performance is assessed and compared against various state-of-the-art machine and deep learning baselines over several popular evaluation metrics. Evaluation on multiple datasets certify the HMA's validity and superiority in ICS anomaly detection, with average AUROC gains of 9.92%, 18.92%, 15.09%, 15.47% and 4.45% with respect to SWaT,

WADI, BATADAL, CISS and CHD datasets. Our future work includes further investigation on enhancing the HMA's runtime efficiency, as well as upgrading the current hierarchical message passing mechanism to generate embeddings incorporating more comprehensive contextual semantics of an ICS network in order to further improve the model's anomaly detection performance.

ACKNOWLEDGMENT

Datasets are provided by iTrust, Centre for Research in Cyber Security, Singapore University of Technology and Design.

REFERENCES

- [1] N. Goldenberg and A. Wool, "Accurate modeling of Modbus/TCP for intrusion detection in SCADA systems," *Int. J. Crit. Infrastructure Protection*, vol. 6, no. 2, pp. 63–75, 2013.
- [2] C. Markman, A. Wool, and A. A. Cardenas, "A new burst-DFA model for SCADA anomaly detection," in *Proc. Workshop Cyber- Phys. Syst. Secur. Privacy*, 2017, pp. 1–12.
- [3] A. Kleinmann and A. Wool, "Automatic construction of statechart-based anomaly detection models for multi-threaded SCADA via spectral analysis," in *Proc. 2nd ACM Workshop Cyber- Phys. Syst. Secur. Privacy*, 2016, pp. 1–12.
- [4] A. Kleinmann and A. Wool, "Automatic construction of statechart-based anomaly detection models for multi-threaded industrial control systems," *ACM Trans. Intell. Syst. Technol.*, vol. 8, no. 4, pp. 1–21, 2017.
- [5] G. Pang, C. Shen, L. Cao, and A. V. D. Hengel, "Deep learning for anomaly detection: A review," *ACM Comput. Surv.*, vol. 54, no. 2, pp. 1–38, 2021.
- [6] M. Abdallah, N. An Le Khac, H. Jahromi, and A. D. Jurcut, "A hybrid CNN-LSTM based approach for anomaly detection systems in SDNS," in *Proc. 16th Int. Conf. Availability, Rel. Secur.*, 2021, pp. 1–7.
- [7] J. Sinha and M. Manollas, "Efficient deep CNN-BiLSTM model for network intrusion detection," in *Proc. 3rd Int. Conf. Artif. Intell. Pattern Recognit.*, 2020, pp. 223–231.
- [8] C. Hwang and T. Lee, "E-SFD: Explainable sensor fault detection in the ICS anomaly detection system," *IEEE Access*, vol. 9, pp. 140470–140486, 2021.
- [9] G. Cao, Y. Wu, D. Yu, and Z. Wang, "Attack detection and location using state forecasting in multivariate time series of ICS," *IEEE Trans. Netw. Sci. Eng.*, vol. 12, no. 4, pp. 2989–3001, Jul.–Aug. 2025.
- [10] P. S. S. G. S. R. E. Selvan, and M. P. Ramkumar, "Anomaly-based intrusion detection system for ICS," in *Proc. 14th Int. Conf. Comput. Commun. Netw. Technol.*, 2023, pp. 1–4.
- [11] Y. Wu, H.-N. Dai, and H. Tang, "Graph neural networks for anomaly detection in industrial Internet of Things," *IEEE Internet Things J.*, vol. 9, no. 12, pp. 9214–9231, Jun. 2022.
- [12] R. R. R. Barbosa, R. Sadre, and A. Pras, "A first look into SCADA network traffic," in *Proc. IEEE Netw. Operations Manage. Symp.*, 2012, pp. 518–521.
- [13] M. R. Asghar, Q. Hu, and S. Zeadally, "Cybersecurity in industrial control systems: Issues, technologies, and challenges," *Comput. Netw.*, vol. 165, 2019, Art. no. 106946.
- [14] M. AlMedires and M. AlMaiah, "Cybersecurity in industrial control system (ICS)," in *Proc. Int. Conf. Inf. Technol.*, 2021, pp. 640–647.
- [15] J. Zhang, S. Gan, X. Liu, and P. Zhu, "Intrusion detection in SCADA systems by traffic periodicity and telemetry analysis," in *Proc. IEEE Symp. Comput. Commun.*, 2016, pp. 318–325.
- [16] M. Caselli, E. Zamboni, and F. Kargl, "Sequence-aware intrusion detection in industrial control systems," in *Proc. 1st ACM Workshop Cyber-Phys. Syst. Secur.*, 2015, pp. 13–24.
- [17] J. Yang, C. Zhou, Y.-C. Tian, and S.-H. Yang, "A software-defined security approach for securing field zones in industrial control systems," *IEEE Access*, vol. 7, pp. 87002–87016, 2019.
- [18] M. A. Umer, K. N. Junejo, M. T. Jilani, and A. P. Mathur, "Machine learning for intrusion detection in industrial control systems: Applications, challenges, and recommendations," *Int. J. Crit. Infrastructure Protection*, vol. 38, 2022, Art. no. 100516.
- [19] A. M. Koay, R. K. L. Ko, H. Hettrema, and K. Radke, "Machine learning in industrial control system (ICS) security: Current landscape, opportunities and challenges," *J. Intell. Inf. Syst.*, vol. 60, pp. 1–29, 2022.
- [20] Z. Deng, X. Zhu, D. Cheng, M. Zong, and S. Zhang, "Efficient KNN classification algorithm for Big Data," *Neurocomputing*, vol. 195, pp. 143–148, 2016.
- [21] C. Zhou and R. C. Paffenroth, "Anomaly detection with robust deep autoencoders," in *Proc. 23rd ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2017, pp. 665–674.
- [22] L. Chen, Y. Li, X. Deng, Z. Liu, M. Lv, and H. Zhang, "Dual auto-encoder GAN-based anomaly detection for industrial control system," *Appl. Sci.*, vol. 12, no. 10, 2022, Art. no. 4986.
- [23] A. Dey, "Deep IDS: A deep learning approach for intrusion detection based on IDS 2018," in *Proc. 2nd Int. Conf. Sustain. Technol. Ind. 4.0*, 2020, pp. 1–5.
- [24] D. Nedeljkovic and Z. Jakovljevic, "CNN based method for the development of cyber-attacks detection algorithms in industrial control systems," *Comput. Secur.*, vol. 114, 2022, Art. no. 102585.
- [25] S. Amraee, A. Vafaei, K. Jamshidi, and P. Adibi, "Abnormal event detection in crowded scenes using one-class SVM," *Signal, Image Video Process.*, vol. 12, no. 6, pp. 1115–1123, 2018.
- [26] L. Abedalla, M. Badarna, W. Khalifa, and M. Yousef, "K-means based one-class SVM classifier," in *Proc. Int. Conf. Database Expert Syst. Appl.*, 2019, pp. 45–53.
- [27] X. Tao, Y. Peng, F. Zhao, P. Zhao, and Y. Wang, "A parallel algorithm for network traffic anomaly detection based on isolation forest," *Int. J. Distrib. Sensor Netw.*, vol. 14, no. 11, 2018, Art. no. 1550147718814471.
- [28] M. U. Togbe et al., "Anomaly detection for data streams based on isolation forest using scikit-multiflow," in *Proc. Int. Conf. Comput. Sci. Appl.*, 2020, pp. 15–30.
- [29] P. Karczmarek, A. Kiersztyn, and W. Pedrycz, "N-ary isolation forest: An experimental comparative analysis," in *Proc. Int. Conf. Artif. Intell. Soft Comput.*, 2020, pp. 188–198.
- [30] C. Yuan and H. Yang, "Research on K-value selection method of K-means clustering algorithm," *J.*, vol. 2, no. 2, pp. 226–235, 2019.
- [31] G. Fortino, C. Greco, A. Guzzo, and M. Ianni, "Neural network based temporal point processes for attack detection in industrial control systems," in *Proc. IEEE Int. Conf. Cyber Secur. Resilience*, 2022, pp. 221–226.
- [32] D. Fährmann, N. Damer, F. Kirchbuchner, and A. Kuijper, "Lightweight long short-term memory variational auto-encoder for multivariate time series anomaly detection in industrial control systems," *Sensors*, vol. 22, no. 8, 2022, Art. no. 2886.
- [33] E. Aboah Boateng and J. Bruce, "Unsupervised machine learning techniques for detecting PLC process control anomalies," *J. Cybersecurity Privacy*, vol. 2, no. 2, pp. 220–244, 2022.
- [34] M. Kravchik and A. Shabtai, "Efficient cyber attack detection in industrial control systems using lightweight neural networks and PCA," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 4, pp. 2179–2197, Jul./Aug. 2022.
- [35] W. Wang et al., "Abnormal detection technology of industrial control system based on transfer learning," *Appl. Math. Computation*, vol. 412, 2022, Art. no. 126539.
- [36] H. T. Truong et al., "Light-weight federated learning-based anomaly detection for time-series data in industrial control systems," *Comput. Ind.*, vol. 140, 2022, Art. no. 103692.
- [37] R. Zhang, Y. Zhang, C. Lu, and X. Li, "Unsupervised graph embedding via adaptive graph learning," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 45, no. 4, pp. 5329–5336, Apr. 2023.
- [38] A. Deng and B. Hooi, "Graph neural network-based anomaly detection in multivariate time series," in *Proc. AAAI Conf. Artif. Intell.*, vol. 35, no. 5, pp. 4027–4035, 2021.
- [39] Z. Chen, D. Chen, X. Zhang, Z. Yuan, and X. Cheng, "Learning graph structures with transformer for multivariate time-series anomaly detection in IoT," *IEEE Internet Things J.*, vol. 9, no. 12, pp. 9179–9189, Jun. 2022.
- [40] S. Han and S. S. Woo, "Learning sparse latent graph representations for anomaly detection in multivariate time series," in *Proc. 28th ACM SIGKDD Conf. Knowl. Discov. Data Mining*, 2022, pp. 2977–2986.
- [41] J. Liang, Z. Du, J. Liang, K. Yao, and F. Cao, "Long and short-range dependency graph structure learning framework on point cloud," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 45, no. 12, pp. 14975–14989, Dec. 2023.
- [42] J. Ouyang, M. Yu, W. Yu, Z. Qin, A. C. Regan, and D. Wu, "TPGraph: A spatial-temporal graph learning framework for accurate traffic prediction on arterial roads," *IEEE Trans. Intell. Transp. Syst.*, vol. 25, no. 5, pp. 3911–3926, May 2024.

- [43] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *Proc. Int. Conf. Learn. Representations*, 2017, pp. 1–14.
- [44] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Lio, and Y. Bengio, "Graph attention networks," in *Proc. Int. Conf. Learn. Representations*, 2018, pp. 1–12.
- [45] W. Hamilton, Z. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, vol. 30, pp. 1025–1035.
- [46] A. Sankar, X. Zhang, and K. C.-C. Chang, "Meta-GNN: Metagraph neural network for semi-supervised learning in attributed heterogeneous information networks," in *Proc. IEEE/ACM Int. Conf. Adv. Social Netw. Anal. Mining*, 2019, pp. 137–144.
- [47] J. Li, Y. Huang, H. Chang, and Y. Rong, "Semi-supervised hierarchical graph classification," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 45, no. 5, pp. 6265–6276, May 2023.
- [48] Z. Zhang et al., "Hierarchical graph pooling with structure learning," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 1, pp. 545–559, Jan. 2023.
- [49] Z. Ying, J. You, C. Morris, X. Ren, W. L. Hamilton, and J. Leskovec, "Hierarchical graph representation learning with differentiable pooling," in *Proc. Adv. Neural Inf. Process. Syst. 31: Annu. Conf. Neural Inf. Process. Syst.* 2018, pp. 4805–4815. [Online]. Available: <https://publications.rwth-aachen.de/record/999809>
- [50] W. W. Lo, S. Layeghy, M. Sarhan, M. Gallagher, and M. Portmann, "E-graphsage: A graph neural network based intrusion detection system for iot," in *Proc. IEEE/IFIP Netw. Operations Manage. Symp.*, 2022, pp. 1–9.
- [51] L. Shuaiyi, K. Wang, L. Zhang, and B. Wang, "Global-local integration for GNN-based anomalous device state detection in industrial control systems," *Expert Syst. Appl.*, vol. 209, 2022, Art. no. 118345.
- [52] X. Deng, J. Zhu, X. Pei, L. Zhang, Z. Ling, and K. Xue, "Flow topology-based graph convolutional network for intrusion detection in label-limited IoT networks," *IEEE Trans. Netw. Service Manag.*, vol. 20, no. 1, pp. 684–696, Mar. 2023.
- [53] Y. Wang, J. Zhang, S. Guo, H. Yin, C. Li, and H. Chen, "Decoupling representation learning and classification for GNN-based anomaly detection," in *Proc. 44th Int. ACM SIGIR Conf. Res. Develop. Inf. Retrieval*, 2021, pp. 1239–1248.
- [54] Q. Do, B. Martini, and K.-K. R. Choo, "The role of the adversary model in applied security research," *Comput. Secur.*, vol. 81, pp. 156–181, 2019. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0167404818306369>
- [55] J. Goh, S. Adepu, K. N. Junejo, and A. Mathur, "A dataset to support research in the design of secure water treatment systems," in *Proc. Int. Conf. Crit. Inf. Infrastructures Secur.*, 2016, pp. 88–99.
- [56] R. Taormina et al., "The battle of the attack detection algorithms: Disclosing cyber attacks on water distribution networks," *J. Water Resour. Plan. Manage.*, vol. 144, no. 8, Aug. 2018, Art. no. 04018048.
- [57] S. L(y)u, K. Wang, L. Zhang, and B. Wang, "Process-oriented heterogeneous graph learning in GNN-based ICS anomalous pattern recognition," *Pattern Recognit.*, vol. 141, no. 109661, pp. 1–14, Sep. 2023.
- [58] Y. Liu et al., "Generative adversarial active learning for unsupervised outlier detection," *IEEE Trans. Knowl. Data Eng.*, vol. 32, no. 8, pp. 1517–1528, Aug. 2020.
- [59] O. Alghushairy, R. Alsini, T. Soule, and X. Ma, "A review of local outlier factor algorithms for outlier detection in Big Data streams," *Big Data Cogn. Comput.*, vol. 5, no. 1, 2020, Art. no. 1.
- [60] M. Tschannen, O. Bachem, and M. Lucic, "Recent advances in autoencoder-based representation learning," in *Proc. NeurIPS 3rd workshop Bayesian Deep Learn.*, 2018, pp. 1–26.
- [61] Y. Wang, J. Wong, and A. Miner, "Anomaly intrusion detection using one class SVM," in *Proc. 5th Annu. IEEE SMC Inf. Assurance Workshop*, 2004, pp. 358–364.
- [62] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation forest," in *Proc. 8th IEEE Int. Conf. Data Mining*, 2008, pp. 413–422.



Shuaiyi Lyu received the MSE degree in electrical and electronic engineering from the University of Pennsylvania, USA, and the PhD degree in cybersecurity from the Harbin Institute of Technology, China. He has authored or coauthored papers in numerous prestigious international journals including *ACM TIST*, *IEEE Transactions on Network Science and Engineering*, and *IEEE Internet of Things Journal*. His research interests include anomaly detection model design and optimization in industrial control systems.



Kai Wang (Member, IEEE) received the PhD degree in communication and information systems from Beijing Jiaotong University, China, in 2014. From 2017 to 2019, he was a postdoc researcher of computer science and technology with Tsinghua University, China. He is currently a full professor with the School of Computer Science and Technology Faculty of Computing, Harbin Institute of Technology, China. He has authored or coauthored more than 40 papers in prestigious international journals, including *IEEE Transactions on Services Computing*, *IEEE Transactions on Intelligent Transportation Systems*, *ACM TOIT*, and *ACM TIST*. His research interests include trustworthy AI and network intrusion detection. He is a member of the ACM, and a senior member of the China Computer Federation.



Yang Liu is currently an associate professor. He has authored or coauthored more than 20 academic papers in journals and conferences both in China and abroad. His research interests include network information security technology and Internet of Things security technology. He has participated in many projects of Ministry of National Information and Science Industry.



Bailing Wang received the PhD degree from the School of Computer Science and Technology, Harbin Institute of Technology, in 2006. He is currently a professor with the Faculty of Computing, Harbin Institute of Technology, China. He has authored or coauthored more than 80 papers in prestigious international journals and conferences. His main research interests include information content security, industrial control network security, and V2X Security. He has been selected for the China national talent plan.